

Memory Mapping

The Problem: Why Can't Processes Share Memory?

Before we learn `mmap()`, we need to understand the problem it solves.

1. Every Process Has Its Own Private Memory

When your program runs, the operating system gives it its own private memory space. This is called a virtual address space. Think of it like each process living in its own separate apartment — they cannot see into each other's rooms.

This means: if you declare a variable `int x = 0` and then call `fork()`, both parent and child have their OWN copy of `x`. Writing to `x` in the child does NOT change the parent's `x`.

This is the problem `mmap()` solves

For process synchronization, processes need a variable they can BOTH see and modify.

A semaphore is just a struct in memory. If each process has its own private copy, `sem_wait()` in one process will never see `sem_post()` from another.

`mmap()` with `MAP_SHARED` creates memory that is physically shared — one location, visible to all processes that map it.

2. What is `mmap()`?

`mmap()` is a system call that asks the operating system to give you a region of memory that is truly shared between processes, not private copies, but the same physical location in RAM.

The signature looks complex but we only use a small part of it in this course:

```
#include <sys/mman.h>

void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);

// Returns: a pointer to the shared memory region
//or MAP_FAILED if something went wrong
```

2.1 The Arguments

Argument	What to pass	Why
<code>addr</code>	<code>NULL</code>	Let the OS choose the address, always do this
<code>length</code>	<code>sizeof(your data)</code>	How many bytes of shared memory you need
<code>prot</code>	<code>PROT_READ PROT_WRITE</code>	You need to both read and write the shared memory
<code>flags</code>	see Section 3 & 4	This controls the most important behaviour — shared vs private, anonymous vs named
<code>fd</code>	<code>-1</code> or <code>shm_fd</code>	Depends on the pattern you are using (explained in Sections 3 & 4)
<code>offset</code>	<code>0</code>	Always 0 in this course

2.2 Mapping Flags

Flag	Meaning	Use Case
MAP_SHARED	Changes are visible to all processes sharing the mapping (and written back to file)	IPC, shared memory
MAP_PRIVATE	Copy-on-write; changes are local to this process; original file unchanged	Loading executables
MAP_ANONYMOUS	Not backed by a file pure RAM. Must pass fd=-1, offset=0	Anonymous shared memory between fork()ed processes
MAP_FIXED	Force the mapping at exactly addr (dangerous, rarely used)	Advanced / system programming

For any inter-process communication, always use MAP_SHARED

3. Mapping a File for Reading

3.1 Traditional read() vs. mmap()

Aspect	read() / write()	mmap()
Data path	Kernel buffer to user buffer (2 copies)	Direct access (kernel maps pages into user space 0 extra copies)
API	read(fd, buf, n) in a loop	ptr = mmap(...); access ptr[i] directly
Error handling	Must check every read/write return value	Access via pointer; errors come as SIGBUS/SIGSEGV
Random access	Must lseek() then read()	Just dereference ptr + offset
Best for	Streaming large sequential data	Random-access, large files, IPC shared memory

3.2 Step-by-Step: Mapping a File for Reading

1. Open the file with open() to get a file descriptor.
2. Call fstat() to find the file size (needed for mmap length).
3. Call mmap() with PROT_READ, MAP_PRIVATE (read-only view).
4. Access the content via the returned pointer.
5. Call munmap() then close() the file descriptor.

Mapping A File For Reading:

```
#include <stdio.h>

#include <stdlib.h>

#include <fcntl.h>

#include <unistd.h>

#include <sys/mman.h>

#include <sys/stat.h>

int main(int argc, char *argv[]) {
```

```

if (argc < 2) { fprintf(stderr, "Usage: %s <file>\n", argv[0]); exit(1); }

// Step 1: open file
int fd = open(argv[1], O_RDONLY);
if (fd == -1) { perror("open"); exit(1); }

// Step 2: get file size
struct stat sb;
fstat(fd, &sb);
size_t length = sb.st_size;

// Step 3: map file into memory
char *data = mmap(NULL, length, PROT_READ, MAP_PRIVATE, fd, 0);
if (data == MAP_FAILED) { perror("mmap"); exit(1); }

// Step 4: use the pointer just like a char array
printf("First 80 characters:\n%.80s\n", data);

// Count newlines (random access with pointer arithmetic)
int lines = 0;
for (size_t i = 0; i < length; i++)
    if (data[i] == '\n') lines++;
printf("Total lines: %d\n", lines);

// Step 5: unmap and close
munmap(data, length);
close(fd);
return 0;

```

3.3 Writing to a File via mmap()

To modify a file through a mapping: open it with `O_RDWR`, map with `PROT_READ | PROT_WRITE` and `MAP_SHARED`. Any write through the pointer is automatically reflected in the file (the kernel will write dirty pages back to disk).

// write_file_mmap.c — modifying a file through a mapping

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>

```

```
#include <sys/stat.h>

int main() {
    // Open file for read+write, create if needed
    int fd = open("data.txt", O_RDWR | O_CREAT, 0644);

    // Ensure the file is at least 64 bytes
    ftruncate(fd, 64);

    // Map with write access — MAP_SHARED writes go to the file
    char *ptr = mmap(NULL, 64, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

    if (ptr == MAP_FAILED) { perror("mmap"); exit(1); }

    // Writing through the pointer modifies the file directly
    strcpy(ptr, "Hello from mmap!");

    munmap(ptr, 64);

    close(fd);

    printf("Written to data.txt via mmap.\n");

    return 0;
}
```

3. Anonymous Shared Memory

Use this when: all your processes are created from one program using `fork()`. No file, no name — just a shared region of RAM that parent and children all see.

3.1 How to Create It

```
// Pattern 1 — anonymous shared memory

// Allocate shared memory — anonymous (no file, no name)
int *shared_var = mmap(
    NULL,           // addr let OS choose
    sizeof(int),    // length enough for one integer
    PROT_READ | PROT_WRITE, // prot read + write
    MAP_SHARED | MAP_ANONYMOUS, // flags shared, no file
    -1,             // fd must be -1 for anonymous
    0               // offset always 0
);

if (shared_var == MAP_FAILED) { // Always check!
    perror("mmap");
    exit(1);
}
```

```
*shared_var = 0; // Use it like a normal pointer
```

3.2 The Key Rules for Pattern 1

- **MAP_ANONYMOUS:** MAP_ANONYMOUS means: no file is involved. The OS just gives you a region of RAM.
- **fd = -1:** Because it is anonymous, fd must be -1 and offset must be 0. These are required.
- **MAP_SHARED:** MAP_SHARED means all processes that map it share the same physical bytes.
- **Before fork():** Call mmap() BEFORE fork() — the child inherits the mapping automatically.

3.3 What Happens After fork()

```
// After fork() — both processes see the same memory

*shared_var = 0;

pid_t pid = fork();

if (pid == 0) {
    // CHILD — writes to shared memory

    *shared_var = 42;

    exit(0);
}

// PARENT — waits then reads

wait(NULL);

printf("%d\n", *shared_var); // prints 42 — because it is truly shared

// Cleaning up anonymous mapping

munmap(shared_var, sizeof(int)); // Release the shared region

// That's it for anonymous — no file, nothing to unlink
```

After fork(), both parent and child have a pointer (shared_var) that points to the same physical location in RAM.

When the child writes *shared_var = 42, the parent sees 42, there is only one copy.

- **munmap:** munmap() takes the same pointer and the same size you gave to mmap().
- **No unlink needed:** For anonymous mappings, there is no file, so nothing else to clean up.

4. Named Shared Memory (shm_open)

Use this when: two completely separate programs, compiled separately, run separately, need to share memory. They cannot use fork() to inherit a mapping, so they both open the same named shared memory object.

The Idea

Think of it like a shared whiteboard with a name written on it. One program creates the whiteboard, another program opens it using the same name. Both then write and read from the same physical surface.

Step	What happens
1. ping.c calls shm_open("/my_shm", O_CREAT O_RDWR, 0666)	Creates a named shared memory object in the kernel. Like creating a new file.
2. ping.c calls ftruncate(fd, size)	Sets the size. shm_open creates a zero-size object — you must set its size before mapping.
3. ping.c calls mmap(MAP_SHARED, fd, 0)	Maps it into ping's address space. Now ping has a pointer to the shared region.

4. pong.c calls <code>shm_open("/my_shm", O_RDWR, 0666)</code>	Opens the same named object. No <code>O_CREAT</code> — it already exists.
5. pong.c calls <code>mmap(MAP_SHARED, fd, 0)</code>	Maps it into pong's address space. Now pong has a pointer to the same physical memory.
6. Either program calls <code>shm_unlink("/my_shm")</code>	Deletes the kernel object when done. Only call this ONCE — pong.c does this in your code.

ping.c:

```
// ping.c: creating and mapping shared memory

#include <fcntl.h>

#include <sys/mman.h>

#include <sys/stat.h>

// Step 1: create the named shared memory object

int shm_fd = shm_open("/shm_pingpong", O_CREAT | O_RDWR, 0666);

// Step 2: set its size

ftruncate(shm_fd, sizeof(shared_data_t));

// Step 3: map it into this program's memory

shared_data_t *data = mmap(NULL,
                             sizeof(shared_data_t),
                             PROT_READ | PROT_WRITE,
                             MAP_SHARED,
                             shm_fd,    // fd from shm_open, NOT -1
                             0);

// Now use data->counter normally

data->counter = 0;
```

pong.c

```
// Step 1: open the existing named object , no O_CREAT, no ftruncate

int shm_fd = shm_open("/shm_pingpong", O_RDWR, 0666);

// Step 2: map it , identical call to ping.c

shared_data_t *data = mmap(NULL,
                             sizeof(shared_data_t),
                             PROT_READ | PROT_WRITE,
                             MAP_SHARED,
                             shm_fd,
                             0);

// data->counter is the exact same memory as ping's data->counter

// Writing here is immediately visible in ping, and vice versa

// In BOTH ping.c and pong.c:
```

```
munmap(data, sizeof(shared_data_t)); // Unmap from this process

close(shm_fd); // Close the file descriptor

// In pong.c ONLY (the last program to run):

shm_unlink("/shm_pingpong"); // Delete the kernel object
```

The Critical Differences Between Creator and Opener

	ping.c (creator)	pong.c (opener)
shm_open flags	O_CREAT O_RDWR	O_RDWR only
ftruncate	Required, sets the size	do not call , size is already set
mmap call	identical in both	identical in both
shm_unlink	do not call , pong does it	call this , you run last